Week 4 – Docker & Docker Compose

Davine Technologies DevOps Internship | Light Osita-Amaechi

1. What is Docker?

Docker is a containerization platform that packages an application together with everything it needs to run – code, runtime, system libraries, and configuration – into a single unit called a container. This solves the classic "it works on my machine" problem, since a container runs the same way on a laptop, a test server, or in production, regardless of what is or isn't installed on the host.

The key difference from a virtual machine is that a container shares the host machine's OS kernel instead of virtualizing an entire operating system. This makes containers far lighter and faster to start than VMs, which need to boot a full guest OS. In this task, I confirmed a working Docker installation by running `docker run hello-world`, which pulled a test image from Docker Hub, ran it in a container, and printed a confirmation message.

2. Docker Architecture

Docker uses a client-server architecture with four main parts. The **Docker client** is the `docker` command-line tool used to issue instructions. The **Docker daemon (dockerd)** is the background service that does the actual work – building images, starting and stopping containers, and managing networks and volumes. The **registry** (Docker Hub by default) stores and distributes images. **Docker objects** – images, containers, volumes, and networks – are the things the daemon manages.

In practice, when I ran `docker run -d --name my-nginx -p 8080:80 nginx:1.27`, the client sent the request to the daemon, the daemon checked for the image locally, and since it wasn't present it pulled `nginx:1.27` from Docker Hub before creating and starting the container.

3. Images vs Containers

An **image** is a read-only, immutable template used to create containers – comparable to a class in programming, or a recipe. A **container** is a running (or stopped) instance created from an image – comparable to an object instantiated from that class, or an actual cake baked from the recipe. Many containers can be created from a single image, and deleting a container does not affect the image it came from.

I demonstrated this directly: running `docker run` multiple times from the same `nginx:1.27` image produced separate, independent containers (`my-nginx`, `web1`, `vol-test`), each with its own container ID, while the underlying image stayed the same.

4. Dockerfile

A Dockerfile is a text file containing instructions Docker executes, in order, to build a custom image. I built the following Dockerfile to serve a custom HTML page on top of nginx:

```
FROM nginx:1.27-alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

`FROM` sets the base image to build on top of – I used the Alpine variant of nginx because it is a minimal Linux distribution, which brought the final image size down to about 73.6 MB versus 282 MB for the standard nginx image. `COPY` copies a file from the build context (the project folder) into the image, creating a new layer. `EXPOSE` documents which port the container listens on but does not publish it – the port still has to be mapped explicitly with `-p` at run time. I built the image with `docker build -t my-custom-app:1.0 .` and confirmed it worked by running it and loading the custom page in a browser at `localhost:8081`.

5. Docker Volumes

A container's filesystem is ephemeral by default – anything written inside it is lost when the container is removed. Volumes are storage managed by Docker that exist independently of any single container's lifecycle, used for data that needs to persist (databases, uploads, etc.).

I created a named volume with `docker volume create app-data` and mounted it into a container with `-v app-data:/usr/share/nginx/html`. To prove persistence, I wrote content into the mounted path, deleted the container entirely, started a brand new container mounting the same volume, and confirmed the content was still there – showing the data lived in the volume, not in the container itself.

6. Docker Networks

By default, containers on Docker's default bridge network cannot reliably reach each other by name. Creating a custom network gives attached containers automatic DNS resolution, meaning one container can reach another simply by using its container name as a hostname – this is the mechanism that makes multi-container applications work.

I created a custom network with `docker network create my-network` and attached two containers to it. From inside one container, running `getent hosts web1` resolved the other container's name to its internal IP address, and `curl web1` successfully returned its web page – confirming both name resolution and connectivity over the custom network, using only the container name and no IP address or port mapping.

7. Docker Compose

Docker Compose defines and runs multi-container applications from a single YAML file, instead of manually running several `docker run` commands. Each service in the file (for example, frontend, backend, and database) is defined with its image, ports, volumes, and network, and Compose creates a shared network by default so services can reach each other by service name – the same name-resolution behaviour proven manually in the Networks section above.

Running `docker compose up -d` starts every defined service together, and `docker compose ps` shows the status of all of them at once, making it far easier to manage a multi-container application than starting each container individually.

8. Docker Hub

Docker Hub is the default public registry for storing and distributing Docker images, functioning similarly to GitHub but for container images. After building my custom image, I authenticated with `docker`

`login`, tagged the image with my Docker Hub namespace using `docker tag my-custom-app:1.0 lightosita/my-custom-app:1.0`, and uploaded it with `docker push`.

Only the layer I added (the copied HTML file) needed to be uploaded, since the base `nginx:1.27-alpine` layers already existed publicly on Docker Hub – a direct example of how Docker's layer caching reduces both build and transfer time. The image is now available at `hub.docker.com/r/lightosita/my-custom-app`.

Submitted as part of Week 4 – Docker & Docker Compose, Davine Technologies DevOps Internship.